Exercise 1 - Dataset Understanding

Topics: Dataset Structure, Scenario Understanding, File Integration, CSV Export

Dataset Folder

In this exercise, you will work with the CMAPSS dataset as a folder that contains multiple files.

Dataset path:
`datasets/CMAPSSData`

Download link:
Download

Goal

In this exercise, your goal is limited to two things:

- understand the dataset structure clearly
- combine all training files into one CSV file and all test files into another CSV file

---

Folder Contents

The dataset folder contains documentation files and data files for four scenarios.

```
datasets/CMAPSSData
│
├── Damage Propagation Modeling.pdf
├── readme.txt
├── RUL_FD001.txt
├── RUL_FD002.txt
├── RUL_FD003.txt
├── RUL_FD004.txt
├── test_FD001.txt
├── test_FD002.txt
├── test_FD003.txt
```

```
├── test_FD004.txt
├── train_FD001.txt
├── train_FD002.txt
├── train_FD003.txt
└── train_FD004.txt
```

What You Need to Understand First

The CMAPSS dataset is not one file. It is a collection of files.

File Type	Role
`train_FDxxx.txt`	training trajectories
`test_FDxxx.txt`	test trajectories
`RUL_FDxxx.txt`	RUL labels for the test engines
`readme.txt`	dataset description
`Damage Propagation Modeling.pdf`	reference document

Scenario Summary

The CMAPSS dataset is divided into four scenarios: FD001, FD002, FD003, and FD004. Each scenario is a different version of the same prediction problem.

Dataset	Train Trajectories	Test Trajectories	Conditions	Fault Modes
FD001	100	100	One	One
FD002	260	259	Six	One
FD003	100	100	One	Two
FD004	248	249	Six	Two

What do these terms mean?

Train Trajectories
This is the number of engine histories available in the training set. A trajectory means all rows that belong to one engine from its first cycle until its last recorded cycle.

Test Trajectories
This is the number of engine histories available in the test set. These engines also have multiple rows, but their data stops before failure.

Conditions
This tells you how many operating settings or operating environments appear in that scenario. A scenario with one condition is simpler. A scenario with six conditions contains more variation.

Fault Modes
This tells you how many degradation patterns are present in that scenario. One fault mode means one type of degradation pattern. Two fault modes means the data contains more than one degradation pattern.

Simple Example: What Is a Trajectory?

Look at the following simplified example:

```
unit  cycle  op1   op2   op3   sensor1  sensor2  ...
1     1      ...   ...   ...   ...      ...
1     2      ...   ...   ...   ...      ...
1     3      ...   ...   ...   ...      ...
1     4      ...   ...   ...   ...      ...

2     1      ...   ...   ...   ...      ...
2     2      ...   ...   ...   ...      ...
2     3      ...   ...   ...   ...      ...
```

In this example:

- all rows with `unit = 1` belong to one engine trajectory
- all rows with `unit = 2` belong to another engine trajectory
- each new cycle gives a new row for the same engine

So when the table says:

- **FD001 has 100 train trajectories** → the training file contains data for 100 engines
- **FD001 has 100 test trajectories** → the test file contains data for 100 engines

---

Why Are the Scenarios Different?

The four scenarios are not copies of each other. They differ in difficulty.

- some scenarios have one operating condition
- some scenarios have six operating conditions
- some scenarios contain one degradation pattern
- some scenarios contain two degradation patterns

Because of that, some scenarios are simpler and some are more complex.

Dataset Structure

Each row is one snapshot for one engine at one cycle.

Use the following column names:

```
unit, cycle, op1, op2, op3, sensor1, sensor2, ..., sensor21
```

In this exercise, list the columns only. Do not explain their physical meaning.

Step 1: Inspect the Folder

List the files inside the dataset folder and identify which ones belong to training, testing, labels, and documentation.

```
import os

DATA_PATH = "datasets/CMAPSSData"

files = os.listdir(DATA_PATH)
for f in files:
    print(f)
```

Answer these questions:

- How many files are inside the folder?
- Which files are training files?
- Which files are test files?

- Which files are RUL files?
- Which files are documentation files?

Step 2: Define the Column Names

Create the full list of columns that will be used when reading the files.

```
columns = ["unit", "cycle", "op1", "op2", "op3"]

for i in range(1, 22):
    columns.append(f"sensor{i}")

print(columns)
print(len(columns))
```

Answer these questions:

- How many total columns are used?
- How many sensor columns are included?
- Why do the text files need manual column names?

Step 3: Define the Training and Test Files

Create one list for all training files and one list for all test files.

```
train_files = [
    "train_FD001.txt",
    "train_FD002.txt",
    "train_FD003.txt",
    "train_FD004.txt"
]

test_files = [
    "test_FD001.txt",
    "test_FD002.txt",
    "test_FD003.txt",
    "test_FD004.txt"
]
```

Answer these questions:

- How many training files are there?

- How many test files are there?
- Why should training and test files remain in separate combined outputs?

Step 4: Load All Training Files and Tag Their Scenario

Read every training file, assign the columns, and add a new column named `scenario`.

```
import pandas as pd

train_dfs = []

for file_name in train_files:
    file_path = os.path.join(DATA_PATH, file_name)
    df = pd.read_csv(file_path, sep=r"\s+", header=None)
    df = df.iloc[:, :26]
    df.columns = columns

    scenario_name = file_name.replace("train_", "").replace("
    df["scenario"] = scenario_name

    train_dfs.append(df)
```

Answer these questions:

- Why is the `scenario` column important?
- Why should you not merge the files without keeping the scenario source?

Step 5: Combine All Training Files into One CSV

Concatenate all training dataframes and save them into one CSV file.

```
train_all_df = pd.concat(train_dfs, ignore_index=True)

train_all_df.to_csv("cmapss_train_all.csv", index=False)

print(train_all_df.shape)
train_all_df.head()
```

Answer these questions:

- What is the name of the final training CSV file?
- How many rows does the merged training dataset contain?
- How many columns does it contain after adding `scenario`?

Step 6: Load All Test Files and Tag Their Scenario

Read every test file, assign the columns, and add the `scenario` column.

```
test_dfs = []

for file_name in test_files:
    file_path = os.path.join(DATA_PATH, file_name)
    df = pd.read_csv(file_path, sep=r"\s+", header=None)
    df = df.iloc[:, :26]
    df.columns = columns

    scenario_name = file_name.replace("test_", "").replace(".
    df["scenario"] = scenario_name

    test_dfs.append(df)
```

Answer these questions:

- Why do you repeat the same process for the test files?
- Why must the test data keep the same column structure as the training data?

Step 7: Combine All Test Files into One CSV

Concatenate all test dataframes and save them into one CSV file.

```
test_all_df = pd.concat(test_dfs, ignore_index=True)

test_all_df.to_csv("cmapss_test_all.csv", index=False)

print(test_all_df.shape)
test_all_df.head()
```

Answer these questions:

- What is the name of the final test CSV file?
- How many rows does the merged test dataset contain?
- Why should the training and test combined files be saved separately?

Step 8: Inspect the Two Combined Files

Check the structure of both merged datasets.

```
print(train_all_df.info())
print(test_all_df.info())
```

```
print(train_all_df["scenario"].value_counts())
print(test_all_df["scenario"].value_counts())
```

Answer these questions:

- Do both merged datasets have the expected columns?
- Does each scenario appear in the final merged files?
- Why is this check necessary before moving to the next exercise?

Step 9: Write a Short Prompt-Based Summary

Do not write code for this part. Write a short text summary using the following points only:

Your summary should include:

- the dataset path
- the number of scenarios
- the names of the combined output files
- the difference between training files and test files
- the list of columns used
- why the `scenario` column was added

Answer these questions in your summary:

- What does this folder contain?

- What did you combine?
- Why were two final CSV files created instead of one?

Expected Output Files

- `cmapss_train_all.csv`
- `cmapss_test_all.csv`
- Your notebook or Python script
- Your short written summary